topic_04_thread_pool

Theory, Edge Cases & Code Examples

- ✓ What is a Thread Pool?
- ✓ Design Decisions
- ✓ **2.1 Task Representation**
- ✓ **2.2 Queue Type**
- ✓ **2.3 Thread Count**

→ Swipe to learn →

topic_04_thread_pool

Theory

📌 1. What is a Thread Pool?

A **thread pool** is a collection of pre-created worker threads that execute tasks from a shared queue.

Benefits:

- ✅ **Avoid thread creation overhead** (threads are expensive to create/destroy)
- ✅ **Limit concurrency** (control max number of active threads)
- ✅ **Better resource utilization** (reuse threads for multiple tasks)
- ✅ **Task scheduling** (prioritize important work)

Typical architecture:

```
        ┌──────────────┐
        │  Task Queue  │
        └──────────────┘
              │
        ┌─────┴─────┐
        │           │
        ▼           ▼
┌──────────────┐ ┌──────────────┐
│  Worker 1    │ │  Worker 2    │ ...
│ (Thread)     │ │ (Thread)     │
└──────────────┘ └──────────────┘
```

Worker loop:

```
while (pool_running):
    task = queue.pop()  // Blocking wait
    task.execute()
```

📌 2. Design Decisions

topic_04_thread_pool

Theory

📌 **2.1 Task Representation**

Option 1: `std::function<void()>`

```
using Task = std::function<void()>;
```

- Simple, type-erased
- No return value

Option 2: `std::packaged_task`

```
using Task = std::packaged_task<void()>;
```

- Can return values via `std::future`
- Supports exceptions

Option 3: Custom callable wrapper

```
template<typename F>
struct Task {
    F func;
    void operator()() { func(); }
};
```

We'll use `std::function` for simplicity, with `std::future` support via `std::packaged_task`.

---

topic_04_thread_pool

Theory

📌 **2.2 Queue Type**

Unbounded queue:
- Tasks can accumulate without limit
- Risk of memory exhaustion

Bounded queue:
- Fixed capacity
- `submit()` blocks when full
- Prevents overload

We'll implement bounded queue (Topic 2 reused).

---

topic_04_thread_pool

Theory

📌 **2.3 Thread Count**

Fixed pool:
- Create N threads at construction
- Simple, predictable

Dynamic pool:
- Adjust thread count based on load
- More complex (thread creation/destruction logic)

We'll implement fixed pool (simpler).

---

topic_04_thread_pool

Theory

📌 **2.4 Shutdown Strategy**

Graceful shutdown:

1. Stop accepting new tasks
2. Wait for queued tasks to finish
3. Join all worker threads

Immediate shutdown:

1. Stop accepting new tasks
2. Discard queued tasks
3. Join all worker threads

We'll implement both.

---

topic_04_thread_pool

Theory

📌 3. Thread Pool Components

1. Task Queue:
- Thread-safe bounded queue (from Topic 2)
- Holds `std::function<void()>` tasks

2. Worker Threads:
- Each runs a loop: pop task → execute → repeat
- Block when queue empty

3. Shutdown Signal:
- Atomic flag: `is_running_`
- Workers check this flag

4. Future Support:
- `std::packaged_task` wraps callable
- Returns `std::future` for result

---

topic_04_thread_pool

Theory

📌 4. Exception Handling (1/7)

Uncaught exceptions in tasks will terminate the program (unless caught).

Solutions:

Option 1: Catch in worker loop

```
try {
    task();
} catch (...) {
    // Log or ignore
}
```

Option 2: Propagate via std::future

```
std::packaged_task<void()> pt(task);
auto fut = pt.get_future();
pt();  // Exception stored in future
fut.get();  // Throws here
```

We'll use Option 1 (catch all exceptions to prevent worker death).

```
#include <vector>
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <functional>
#include <future>
#include <stdexcept>
#include <atomic>
#include <iostream>

class ThreadPool {
private:
    // Worker threads
    std::vector<std::thread> workers_;

    // Task queue
    std::queue<std::function<void()>> tasks_;
// ...continues on next slide
```

📌 4. Exception Handling (2/7)

```
// ...continued from previous slide

    // Synchronization
    std::mutex queue_mutex_;
    std::condition_variable condition_;

    // State
    std::atomic<bool> stop_{false};       // Stop accepting new tasks
    std::atomic<bool> terminate_{false};  // Immediate shutdown flag

    // Statistics (optional)
    std::atomic<size_t> active_tasks_{0};
    std::atomic<size_t> total_tasks_{0};

public:
    // ============================================================
    // CONSTRUCTOR
    // ============================================================

    explicit ThreadPool(size_t num_threads = std::thread::hardware_concurrency())
    {
        if (num_threads == 0) {
            throw std::invalid_argument("Thread count must be > 0");
        }

        workers_.reserve(num_threads);

        // Create worker threads
        for (size_t i = 0; i < num_threads; ++i) {
            workers_.emplace_back([this, i]() {
                worker_loop(i);
            });
        }
    }

    // ============================================================
    // DESTRUCTOR - Graceful Shutdown
    // ============================================================

    ~ThreadPool() {
        shutdown();
// ...continues on next slide
```

📌 4. Exception Handling (3/7)

```
// ...continued from previous slide
    }

    // Disable copy/move (managing threads)
    ThreadPool(const ThreadPool&) = delete;
    ThreadPool& operator=(const ThreadPool&) = delete;
    ThreadPool(ThreadPool&&) = delete;
    ThreadPool& operator=(ThreadPool&&) = delete;

    // ============================================================
    // SUBMIT TASK (FIRE-AND-FORGET)
    // ============================================================

    template<typename F, typename... Args>
    void submit(F&& f, Args&&... args) {
        if (stop_.load(std::memory_order_acquire)) {
            throw std::runtime_error("Cannot submit to stopped pool");
        }

        {
            std::lock_guard<std::mutex> lock(queue_mutex_);

            // Bind arguments to create zero-argument callable
            tasks_.emplace([f = std::forward<F>(f),
                            ...args = std::forward<Args>(args)]() mutable {
                f(std::forward<Args>(args)...);
            });

            total_tasks_.fetch_add(1, std::memory_order_relaxed);
        }

        // Wake one waiting worker
        condition_.notify_one();
    }

    // ============================================================
    // SUBMIT TASK (WITH FUTURE FOR RESULT)
    // ============================================================

    template<typename F, typename... Args>
    auto submit_with_result(F&& f, Args&&... args)
// ...continues on next slide
```

📌 4. Exception Handling (4/7)

```
// ...continued from previous slide
        -> std::future<std::invoke_result_t<F, Args...>>
    {
        using ReturnType = std::invoke_result_t<F, Args...>;

        if (stop_.load(std::memory_order_acquire)) {
            throw std::runtime_error("Cannot submit to stopped pool");
        }

        // Create packaged_task with bound arguments
        auto task = std::make_shared<std::packaged_task<ReturnType()>>(
            std::bind(std::forward<F>(f), std::forward<Args>(args)...)
        );

        std::future<ReturnType> result = task->get_future();

        {
            std::lock_guard<std::mutex> lock(queue_mutex_);

            tasks_.emplace([task]() {
                (*task)();
            });

            total_tasks_.fetch_add(1, std::memory_order_relaxed);
        }

        condition_.notify_one();

        return result;
    }

    // ============================================================
    // SHUTDOWN
    // ============================================================

    // Graceful shutdown: finish queued tasks
    void shutdown() {
        {
            std::lock_guard<std::mutex> lock(queue_mutex_);
            if (stop_.load(std::memory_order_relaxed)) {
                return;  // Already stopped
// ...continues on next slide
```

📌 4. Exception Handling (5/7)

```
// ...continued from previous slide
            }
            stop_.store(true, std::memory_order_release);
        }

        // Wake all workers
        condition_.notify_all();

        // Join all threads
        for (auto& worker : workers_) {
            if (worker.joinable()) {
                worker.join();
            }
        }
    }

    // Immediate shutdown: discard queued tasks
    void shutdown_now() {
        {
            std::lock_guard<std::mutex> lock(queue_mutex_);
            if (terminate_.load(std::memory_order_relaxed)) {
                return;
            }

            stop_.store(true, std::memory_order_release);
            terminate_.store(true, std::memory_order_release);

            // Clear task queue
            std::queue<std::function<void()>> empty;
            std::swap(tasks_, empty);
        }

        condition_.notify_all();

        for (auto& worker : workers_) {
            if (worker.joinable()) {
                worker.join();
            }
        }
    }

// ...continues on next slide
```

📌 4. Exception Handling (6/7)

```
// ...continued from previous slide
    // ============================================================
    // QUERY OPERATIONS
    // ============================================================

    size_t thread_count() const {
        return workers_.size();
    }

    size_t queued_tasks() const {
        std::lock_guard<std::mutex> lock(queue_mutex_);
        return tasks_.size();
    }

    size_t active_tasks() const {
        return active_tasks_.load(std::memory_order_relaxed);
    }

    size_t total_tasks() const {
        return total_tasks_.load(std::memory_order_relaxed);
    }

    bool is_stopped() const {
        return stop_.load(std::memory_order_acquire);
    }

private:
    // ============================================================
    // WORKER LOOP (RUNS IN EACH THREAD)
    // ============================================================

    void worker_loop(size_t worker_id) {
        while (true) {
            std::function<void()> task;

            {
                std::unique_lock<std::mutex> lock(queue_mutex_);

                // Wait for task or shutdown
                condition_.wait(lock, [this]() {
                    return !tasks_.empty() ||
// ...continues on next slide
```

topic_04_thread_pool

Theory

📌 4. Exception Handling (7/7)

[Code continues across slides]

```
// ...continued from previous slide
                            stop_.load(std::memory_order_relaxed);
                });

                // Exit if stopped and no tasks remain
                if (stop_.load(std::memory_order_relaxed)) {
                    if (tasks_.empty() || terminate_.load(std::memory_order_relaxed)) {
                        return;  // Exit thread
                    }
                }

                // Pop task
                if (!tasks_.empty()) {
                    task = std::move(tasks_.front());
                    tasks_.pop();
                }
            }

            // Execute task (outside lock)
            if (task) {
                active_tasks_.fetch_add(1, std::memory_order_relaxed);

                try {
                    task();
                } catch (const std::exception& e) {
                    std::cerr << "Worker " << worker_id
                              << " caught exception: " << e.what() << '\n';
                } catch (...) {
                    std::cerr << "Worker " << worker_id
                              << " caught unknown exception\n";
                }

                active_tasks_.fetch_sub(1, std::memory_order_relaxed);
            }
        }
    }
};
```

topic_04_thread_pool

Edge Cases

📌 1. Submit After Shutdown

Problem: Calling `submit()` after `shutdown()` may crash.

```
pool.shutdown();
pool.submit([]() { /* ... */ });  // ← Should throw
```

Solution: Check `stop_` flag:

```
if (stop_.load(std::memory_order_acquire)) {
    throw std::runtime_error("Cannot submit to stopped pool");
}
```

---

topic_04_thread_pool

Edge Cases

📌 2. Exception in Task

Problem: Uncaught exception terminates worker thread.

```
pool.submit([]() {
    throw std::runtime_error("Task failed");
    // Worker dies, pool has fewer threads!
});
```

Solution: Catch all exceptions in worker loop:

```
try {
    task();
} catch (const std::exception& e) {
    std::cerr << "Caught: " << e.what() << '\n';
} catch (...) {
    std::cerr << "Unknown exception\n";
}
```

---

topic_04_thread_pool

Edge Cases

📌 3. Deadlock on Shutdown

Problem: Worker waits on condition variable, shutdown doesn't notify.

```
void shutdown() {
    stop_ = true;
    // Forgot: condition_.notify_all();
    for (auto& t : workers_) t.join();  // Hangs forever
}
```

Solution: Always notify after setting stop flag:

```
stop_.store(true, std::memory_order_release);
condition_.notify_all();  // Wake all workers
```

---

topic_04_thread_pool

Edge Cases

📌 4. Double Shutdown

Problem: Calling `shutdown()` twice may cause issues.

```
pool.shutdown();
pool.shutdown();  // ← May hang or crash
```

Solution: Check if already stopped:

```
void shutdown() {
    if (stop_.load(std::memory_order_relaxed)) {
        return;  // Already stopped
    }
    // ... shutdown logic ...
}
```

---

topic_04_thread_pool

Edge Cases

📌 5. Task Captures Reference to Stack Variable

Problem: Task outlives captured variable.

```
void foo() {
    ThreadPool pool(4);
    int x = 42;

    pool.submit([&x]() {
        std::cout << x << '\n';  // ← DANGER: x may be destroyed
    });

    // Function returns before task executes → x destroyed → UB
}
```

Solution:

- Capture by value: `[x]`
- Or use `shared_ptr`: `[ptr = std::make_shared<int>(42)]`

---

topic_04_thread_pool

Code Examples

📌 1. Basic Task Submission (1/2)

This example demonstrates core thread pool functionality: distributing tasks across a fixed number of worker threads for parallel execution.

What this code does:
- Creates thread pool with 4 worker threads
- Submits 10 tasks (more tasks than threads, demonstrating queuing)
- Each task prints its ID and the thread executing it, then sleeps 100ms
- Tasks execute concurrently (4 at a time), remaining tasks queue
- Main thread waits 1 second for completion, then graceful shutdown

Key concepts demonstrated:
- **Thread reuse**: Same 4 threads execute all 10 tasks (no thread creation overhead per task)
- **Task queuing**: When all workers busy, tasks wait in queue (FIFO order)
- **Concurrency**: Output interleaves showing true parallelism (4 tasks run simultaneously)
- **Graceful shutdown**: Destructor waits for queued tasks to complete before joining threads
- **Fire-and-forget submission**: submit() returns immediately, task executes asynchronously

Real-world applications:
- Web servers handling multiple HTTP requests with limited threads
- Image processing pipeline (resize, compress, watermark operations)
- Database connection pools executing queries
- Background job processors (email sending, report generation)
- Parallel test execution frameworks

Why this matters:
- **Resource control**: Limit concurrent threads to prevent oversubscription (CPU cores limit)
- **Better than std::async**: async may create unbounded threads; thread pool controls max concurrency
- **Improved latency**: Avoid thread creation cost (~50-100μs) per task
- **Fairness**: FIFO queue ensures tasks processed in submission order

Performance implications:
- Thread creation avoided: 10 tasks, 0 thread creations (vs 10 with std::thread per task)
- Context switching: Only 4 threads compete for CPU (vs potential 10+ with naive approach)
- Memory footprint: 4 × ~1MB stack = 4MB (vs 10MB for thread-per-task)

Output (interleaved):

---

topic_04_thread_pool

Code Examples

📌 1. Basic Task Submission (2/2)

```
#include <iostream>
#include <chrono>

void task(int id) {
    std::cout << "Task " << id << " starting on thread "
              << std::this_thread::get_id() << '\n';

    std::this_thread::sleep_for(std::chrono::milliseconds(100));

    std::cout << "Task " << id << " completed\n";
}

int main() {
    ThreadPool pool(4);  // 4 worker threads

    // Submit 10 tasks
    for (int i = 0; i < 10; ++i) {
        pool.submit(task, i);
    }

    // Wait for all tasks to complete
    std::this_thread::sleep_for(std::chrono::seconds(1));

    pool.shutdown();

    std::cout << "Total tasks processed: " << pool.total_tasks() << '\n';

    return 0;
}
```

```
['Task 0 starting on thread 140234567\nTask 1 starting on thread 140234568\nTask 2 starting on thread 140234569\nTask 3
```

topic_04_thread_pool

Code Examples

📌 2. Using Futures for Results (1/2)

This example demonstrates returning values from thread pool tasks using std::future for synchronization and result retrieval.

What this code does:
- Submits 10 parallel tasks, each computing sum of 1000-element range
- `submit_with_result()` returns `std::future<int>` for each task
- Main thread collects results by calling `fut.get()` (blocks until result ready)
- Aggregates partial sums into final total
- Demonstrates map-reduce pattern: parallel computation + sequential reduction

Key concepts demonstrated:
- **std::packaged_task**: Wraps callable with future, stores result/exception
- **Synchronization via futures**: `fut.get()` blocks until task completes (implicit join)
- **Exception propagation**: If task throws, exception stored in future and rethrown on `get()`
- **Type safety**: Future's template parameter ensures type-correct result retrieval
- **Parallel map-reduce**: Divide work across threads, combine results in main thread

Real-world applications:
- Parallel numerical computations (matrix operations, Monte Carlo simulations)
- Distributed search (search different parts of dataset in parallel)
- Parallel file processing (parse multiple files, aggregate results)
- Concurrent HTTP requests (fetch from multiple APIs, combine responses)
- Divide-and-conquer algorithms parallelization

Why this matters:
- **Clean result handling**: Futures provide type-safe result passing without shared state
- **Automatic synchronization**: No need for manual mutexes/condition variables
- **Exception safety**: Errors propagate cleanly without crashing worker threads
- **Scalability**: Easily parallelize embarrassingly parallel workloads

Performance implications:
- Speedup: Ideally ~4× on 4-core CPU (if tasks are CPU-bound and equal-sized)
- Actual speedup: Limited by Amdahl's law (sequential `get()` calls and aggregation)
- Memory: Each future holds result, so 10 futures = 10 ints (~40 bytes)
- Cache effects: Better locality if ranges are contiguous (less cache thrashing)

Output:

---

topic_04_thread_pool

Code Examples

📌 2. Using Futures for Results (2/2)

```
#include <iostream>
#include <numeric>
#include <vector>

int sum_range(int start, int end) {
    int sum = 0;
    for (int i = start; i < end; ++i) {
        sum += i;
    }
    return sum;
}

int main() {
    ThreadPool pool(4);

    std::vector<std::future<int>> futures;

    // Submit 10 parallel sum tasks
    for (int i = 0; i < 10; ++i) {
        futures.push_back(
            pool.submit_with_result(sum_range, i * 1000, (i + 1) * 1000)
        );
    }

    // Collect results
    int total = 0;
    for (auto& fut : futures) {
        total += fut.get();  // Blocks until result ready
    }

    std::cout << "Total sum: " << total << '\n';

    pool.shutdown();

    return 0;
}
```

```
['Total sum: 4995000']
```

topic_04_thread_pool

Code Examples

📌 3. Parallel Matrix Multiplication (1/3)

This example demonstrates parallelizing matrix multiplication by distributing row computations across thread pool workers.

What this code does:
- Multiplies two 2×2 matrices (A × B = C) using thread pool
- Each row of result matrix C computed by separate task
- Creates future for each row computation (2 futures for 2×2 matrix)
- Main thread waits for all row computations via `fut.get()`
- Result matrix built collaboratively by parallel workers

Key concepts demonstrated:
- **Row-wise parallelization**: Each row independent, perfect for parallel computation
- **std::ref for shared state**: Pass matrices by reference to avoid copying into lambdas
- **Synchronization barrier**: Loop of `get()` calls acts as barrier (all rows must complete)
- **Load balancing**: std::thread::hardware_concurrency() adapts pool size to CPU cores
- **Data parallelism**: Same operation (row multiplication) on different data (different rows)

Real-world applications:
- Scientific computing libraries (BLAS, Eigen with threading)
- Graphics pipelines (transformation matrix chains)
- Neural network inference (parallel layer computations)
- Image processing (convolution operations)
- Physics simulations (force calculations on particle systems)

Why this matters:
- **Scalability**: For large matrices (1000×1000), N-core CPU gives ~N× speedup
- **Flexibility**: Can choose granularity (rows vs blocks vs elements)
- **Cache efficiency**: Row-wise access is cache-friendly (contiguous memory)
- **Easy parallelization**: Minimal code changes from serial implementation

Performance implications:
- Small matrices (2×2): Overhead dominates, serial likely faster
- Large matrices (1000×1000): Thread pool shines, near-linear speedup
- Optimal task size: Rows should be large enough to amortize task submission overhead
- Memory bandwidth: May become bottleneck (matrix data transfer to/from cache)

Optimization considerations:
- For small matrices: Use serial code (overhead exceeds benefit)
- For huge matrices: Block-based parallelization for better cache locality
- SIMD: Combine with vectorization (SSE/AVX) for additional speedup

topic_04_thread_pool

Code Examples

📌 3. Parallel Matrix Multiplication (2/3)

```
#include <vector>
#include <iostream>

using Matrix = std::vector<std::vector<int>>;

void multiply_row(const Matrix& A, const Matrix& B, Matrix& C, int row) {
    int n = B[0].size();
    int k = B.size();

    for (int col = 0; col < n; ++col) {
        C[row][col] = 0;
        for (int i = 0; i < k; ++i) {
            C[row][col] += A[row][i] * B[i][col];
        }
    }
}

Matrix parallel_matmul(const Matrix& A, const Matrix& B) {
    int m = A.size();
    int n = B[0].size();

    Matrix C(m, std::vector<int>(n, 0));

    ThreadPool pool(std::thread::hardware_concurrency());

    std::vector<std::future<void>> futures;

    // Each row computed in parallel
    for (int row = 0; row < m; ++row) {
        futures.push_back(
            pool.submit_with_result(multiply_row,
                                    std::ref(A), std::ref(B), std::ref(C), row)
        );
    }

    // Wait for all rows
    for (auto& fut : futures) {
        fut.get();
    }

    return C;
// ...continues on next slide
```

topic_04_thread_pool

Code Examples

📌 3. Parallel Matrix Multiplication (3/3)

[Code continues across slides]

```
// ...continued from previous slide
}

int main() {
    Matrix A = {{1, 2}, {3, 4}};
    Matrix B = {{5, 6}, {7, 8}};

    Matrix C = parallel_matmul(A, B);

    std::cout << "Result:\n";
    for (const auto& row : C) {
        for (int val : row) {
            std::cout << val << " ";
        }
        std::cout << '\n';
    }

    return 0;
}
```

```
['Result:\n19 22\n43 50']
```

topic_04_thread_pool

Code Examples

📌 4. Immediate Shutdown vs Graceful (1/3)

This example compares graceful shutdown (completes queued tasks) vs immediate shutdown (discards queued tasks) in thread pools.

What this code does:
- Defines long-running task (2-second sleep) to make queue behavior visible
- **Graceful test**: Submits 5 tasks to 2-worker pool, waits 100ms, calls `shutdown()`
- First 2 tasks start immediately
- Remaining 3 queue
- shutdown() waits for all 5 to finish (~10 seconds total)
- **Immediate test**: Same setup but calls `shutdown_now()`
- First 2 tasks complete (already running)
- Remaining 3 discarded
- shutdown_now() returns quickly (~4 seconds)

Key concepts demonstrated:
- **Graceful shutdown semantics**: Honor all submitted work, useful for critical tasks
- **Immediate shutdown semantics**: Abort pending work, useful for cleanup/cancellation
- **Running vs queued**: shutdown_now() cannot stop running tasks (no thread cancellation in C++)
- **Resource cleanup**: Both methods join threads before returning (safe destruction)
- **Policy choice**: Application decides between throughput (graceful) and responsiveness (immediate)

Real-world applications:
- **Graceful**: Web server shutdown (finish in-flight requests), batch job processor
- **Immediate**: Interactive application exit (discard background tasks), timeout handling
- **Hybrid**: Critical tasks use graceful, optional tasks use immediate
- **Staged shutdown**: First try graceful with timeout, fallback to immediate

Why this matters:
- **User experience**: Graceful prevents data loss; immediate prevents hang on exit
- **Resource management**: Immediate releases resources faster (important in tight loops)
- **Testing**: Immediate shutdown reduces test runtime (don't wait for all tasks)
- **Error handling**: Immediate useful when exception detected (abort remaining work)

Performance implications:
- Graceful: Predictable completion (all tasks finish), longer shutdown time
- Immediate: Faster shutdown (~2× in this example), potential wasted work
- Memory: Immediate frees task queue earlier (matters with large queued tasks)

Design decisions:
- Destructor uses graceful shutdown (safe default, prevents task loss)

topic_04_thread_pool

Code Examples

📌 4. Immediate Shutdown vs Graceful (2/3)

```
#include <iostream>
#include <chrono>

void long_task(int id) {
    std::cout << "Task " << id << " started\n";
    std::this_thread::sleep_for(std::chrono::seconds(2));
    std::cout << "Task " << id << " completed\n";
}

void test_graceful_shutdown() {
    std::cout << "=== Graceful Shutdown ===\n";
    ThreadPool pool(2);

    for (int i = 0; i < 5; ++i) {
        pool.submit(long_task, i);
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(100));

    std::cout << "Shutting down gracefully...\n";
    pool.shutdown();  // Waits for all 5 tasks
    std::cout << "All tasks completed\n\n";
}

void test_immediate_shutdown() {
    std::cout << "=== Immediate Shutdown ===\n";
    ThreadPool pool(2);

    for (int i = 0; i < 5; ++i) {
        pool.submit(long_task, i);
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(100));

    std::cout << "Shutting down immediately...\n";
    pool.shutdown_now();  // Only running tasks finish, queued discarded
    std::cout << "Shutdown complete\n\n";
}

int main() {
    test_graceful_shutdown();
// ...continues on next slide
```

topic_04_thread_pool

Code Examples

📌 4. Immediate Shutdown vs Graceful (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    test_immediate_shutdown();

    return 0;
}
```

```
['=== Graceful Shutdown ===\nTask 0 started\nTask 1 started\nShutting down gracefully...\nTask 0 completed\nTask 1 comp
```

What You Learned

Key Takeaways

- ✓ What is a Thread Pool?
- ✓ Design Decisions
- ✓ **2.1 Task Representation**
- ✓ **2.2 Queue Type**

→ Swipe to learn →